[Nazwa uczelni]
[Wydział / kierunek]

Algorytmy inteligentne inspirowane naturą

Dokumentacja projektu

Zastosowanie algorytmu mrówkowego (ACO) do rozwiązania problemu komiwojażera (TSP)

Autorzy: [imię i nazwisko], [imię i nazwisko]
Grupa: [numer grupy]
Prowadzący: [tytuł, imię i nazwisko]
Wersja dokumentu: 2.0 – szczegółowy opis projektu na drugie zajęcia
Data: listopad 2026

Spis treści

Spis treści 2
1. Wprowadzenie 4
1.1. Cel projektu 4
1.2. Uzasadnienie wyboru tematu 4
1.3. Zakres prac 4
1.4. Stan prac 5
2. Opis problemu 6
2.1. Problem komiwojażera 6
2.2. Model planszy 6
3. Algorytm mrówkowy 7
3.1. Klasyfikacja algorytmu 7
3.2. Inspiracja biologiczna 8
3.3. Budowa trasy przez mrówkę 8
3.4. Aktualizacja feromonu 8
3.5. Pseudokod 9
3.6. Parametry algorytmu 9
3.7. Szczegóły implementacji 9
3.8. Dotychczasowe wyniki w literaturze 10
4. Algorytmy porównawcze 11
4.1. Przegląd zupełny (brute force) 11
4.2. Algorytm najbliższego sąsiada 11
4.3. Algorytm 2-opt 11
4.4. Zestawienie 11
5. Implementacja 12
5.1. Technologie 12
5.2. Struktura projektu 12
5.3. Najważniejsze elementy kodu 12
6. Interfejs użytkownika – instrukcja obsługi 14
6.1. Okno aplikacji i sterowanie 14
6.2. Panel „Algorytm i wzory” 16
6.3. Podgląd reguły wyboru (tryb „Wybór”) 17
6.4. Animacja mrówek 19
6.5. Porównanie algorytmów 20
7. Badania i wyniki wstępne 21
7.1. Metodyka 21

7.2. Małe instancje – porównanie z optimum 21
7.3. Duże instancje 22
7.4. Wpływ parametrów 22
7.5. Rozszerzenie: zablokowanie ścieżki 23
7.6. Wnioski wstępne 25
8. Plan dalszych prac 26
Bibliografia 27

1. Wprowadzenie

1.1. Cel projektu

Celem projektu jest zbadanie, jak algorytm mrówkowy (ang. *Ant Colony Optimization*, ACO) radzi sobie z problemem komiwojażera (ang. *Travelling Salesman Problem*, TSP), oraz porównanie go z metodą dokładną i dwiema klasycznymi heurystykami. Problem przedstawiono w formie bliskiej oryginalnej inspiracji: na planszy znajduje się mrowisko oraz punkty z pożywieniem, a kolonia mrówek szuka najkrótszej trasy, która zaczyna się w mrowisku, odwiedza każdy punkt z pożywieniem dokładnie raz i wraca do mrowiska.

Działanie algorytmu jest pokazane w interaktywnej aplikacji: widać, jak w kolejnych iteracjach na ścieżkach odkłada się feromon, jak słabsze ścieżki stopniowo zanikają i jak kolonia skupia się na coraz krótszej trasie.

1.2. Uzasadnienie wyboru tematu

- **Problem jest trudny obliczeniowo.** Problem komiwojażera jest NP-trudny, a liczba możliwych tras rośnie jak n!/2. Już przy kilkunastu punktach przegląd zupełny przestaje być wykonalny, więc potrzebne są metody przybliżone – dokładnie ten obszar, w którym stosuje się algorytmy inspirowane naturą.
- **Problem ma praktyczne zastosowania.** Do TSP sprowadza się planowanie tras dostaw i odbioru odpadów, kolejność wiercenia otworów w płytkach drukowanych, ruch głowicy maszyn CNC czy kolejność zadań w produkcji.
- **Algorytm mrówkowy powstał właśnie dla TSP.** M. Dorigo zaproponował go w 1992 roku i pierwsze wyniki pokazał na problemie komiwojażera. TSP jest do dziś podstawowym problemem testowym dla nowych wariantów ACO, więc łatwo porównać własne wyniki z literaturą.
- **Temat dobrze się wizualizuje.** Plansza z mrowiskiem i pożywieniem odpowiada biologicznej inspiracji algorytmu, a feromon na ścieżkach można rysować bezpośrednio. Dzięki temu działanie algorytmu – w tym wpływ parametrów α, β i ρ – widać, a nie tylko odczytuje się je z liczb.
- **Wyniki można rzetelnie ocenić.** Dla małych plansz znamy trasę optymalną (przegląd zupełny), a dla większych mamy klasyczne heurystyki (najbliższy sąsiad, 2-opt), więc jakość i czas działania ACO da się zmierzyć.

1.3. Zakres prac

- implementacja algorytmu mrówkowego (Ant System ze strategią elitarną),
- implementacja algorytmów porównawczych: przeglądu zupełnego, najbliższego sąsiada i 2-opt,
- aplikacja w bibliotece Pygame do wizualizacji i ręcznego ustawiania planszy, z panelem wzorów, podglądem prawdopodobieństw wyboru i animacją mrówek,
- eksperymenty: długość trasy i czas obliczeń dla rosnącej liczby punktów, wpływ parametrów ACO,

- rozszerzenie opcjonalne: zachowanie kolonii po zablokowaniu jednej ze ścieżek.

1.4. Stan prac

Niniejsza wersja dokumentu opisuje projekt na drugie zajęcia. Wszystkie elementy z punktu 1.3 działają. W porównaniu z wersją wstępną dodano klasyfikację algorytmu i przegląd wyników z literatury (rozdział 3), a aplikację rozbudowano o elementy pokazujące, jak działają wzory algorytmu: panel ze wzorami i bieżącymi parametrami, podgląd prawdopodobieństw wyboru kolejnego punktu oraz animację mrówek (rozdział 6). Wstępne wyniki eksperymentów przedstawiono w rozdziale 7, a plan dalszych prac w rozdziale 8.

2. Opis problemu

2.1. Problem komiwojażera

Dany jest zbiór n + 1 punktów: mrowisko (punkt 0) oraz n punktów z pożywieniem (1, …, n). Dla każdej pary punktów i, j znana jest odległość d_{ij}. Szukamy permutacji π punktów, która zaczyna się w mrowisku i minimalizuje długość zamkniętej trasy:

$$L(\pi) = \sum_{k=0}^{n-1} d_{\pi_k \pi_{k+1}} + d_{\pi_n \pi_0} \rightarrow \min, \qquad \pi_0 = 0 \qquad (1)$$

Odległości są symetryczne ($d_{ij} = d_{ji}$), więc każdą trasę można przejść w dwóch kierunkach. Przy ustalonym punkcie startowym liczba różnych tras wynosi **n!/2**. Dla 10 punktów pożywienia to 1 814 400 tras, a dla 20 punktów już około $1{,}2 \cdot 10^{18}$. Problem komiwojażera jest NP-trudny: nie jest znany algorytm, który zawsze znajduje trasę optymalną w czasie wielomianowym. Dlatego dla większej liczby punktów stosuje się heurystyki i metaheurystyki, takie jak algorytm mrówkowy.

2.2. Model planszy

- Plansza ma wymiary 900 × 760 pikseli; w eksperymentach punkty losuje się w kwadracie 100 × 100.
- Odległość między punktami jest odległością euklidesową.
- Mrowisko jest zawsze punktem startowym i końcowym trasy; można je przesuwać, ale nie można go usunąć.
- Ścieżkę między dwoma punktami można zablokować. Zablokowana krawędź dostaje bardzo dużą długość (10^6), dlatego każdy algorytm jej unika, a trasa, która mimo to z niej korzysta, jest od razu widoczna jako bardzo długa.

3. Algorytm mrówkowy

3.1. Klasyfikacja algorytmu

Algorytm mrówkowy należy do **algorytmów inspirowanych naturą**, a w ich obrębie do grupy **inteligencji roju** (ang. *swarm intelligence*). Algorytmy tej grupy wykorzystują populację prostych agentów, z których żaden nie zna całego problemu, a dobre rozwiązanie wyłania się z ich lokalnych działań i pośredniej komunikacji. Jest to **metaheurystyka populacyjna i konstrukcyjna**: każda mrówka buduje kompletne rozwiązanie od zera, krok po kroku, a „pamięcią" całej kolonii jest macierz feromonu.

```
Algorytmy inspirowane naturą
├── algorytmy ewolucyjne (algorytm genetyczny, strategie ewolucyjne, ...)
├── inteligencja roju                          ◂ ta grupa
│    ├── optymalizacja rojem cząstek (PSO)
│    ├── sztuczna kolonia pszczół (ABC)
│    └── optymalizacja kolonią mrówek (ACO)    ◂ ta rodzina
│         ├── Ant System (AS)
│         ├── Elitist Ant System (EAS)         ◂ wariant użyty w projekcie
│         ├── Rank-based Ant System (ASrank)
│         ├── Ant Colony System (ACS)
│         └── MAX–MIN Ant System (MMAS)
└── inne (wyżarzanie symulowane, systemy immunologiczne, ...)
```

Tabela 1. *Najważniejsze warianty algorytmu mrówkowego dla TSP*

Wariant	Autorzy, rok	Czym się wyróżnia
Ant System (AS)	Dorigo, Maniezzo, Colorni 1991/1996	pierwsza wersja: wszystkie mrówki odkładają feromon proporcjonalny do $1/L_k$
Elitist AS (EAS)	Dorigo 1992, Dorigo i in. 1996	AS + dodatkowy feromon na najlepszej dotąd trasie (waga e) – szybsza zbieżność
Rank-based AS	Bullnheimer, Hartl, Strauss 1999	feromon odkładają tylko najlepsze mrówki, z wagą zależną od miejsca w rankingu
Ant Colony System (ACS)	Dorigo, Gambardella 1997	reguła pseudolosowa, lokalna aktualizacja feromonu w trakcie budowy trasy, feromon odkłada tylko najlepsza mrówka
MAX–MIN AS (MMAS)	Stützle, Hoos 2000	feromon ograniczony z dołu i z góry $[\tau_{min}, \tau_{max}]$, odkłada go tylko najlepsza mrówka – mniejsze ryzyko stagnacji

Wariant użyty w projekcie to Ant System ze strategią elitarną (EAS), uzupełniony o dolne ograniczenie feromonu (zapożyczone z MMAS). Wybraliśmy go, ponieważ jest najbliższy oryginalnej idei – feromon odkładają wszystkie mrówki, więc na planszy widać ślady całej kolonii, a nie tylko jednej trasy – a przy tym dzięki wzmocnieniu najlepszej trasy zbiega szybciej niż podstawowy AS. Proste reguły ułatwiają też pokazanie wzorów na żywo w aplikacji. Warianty ACS i MMAS planujemy wykorzystać w dalszej części projektu jako punkt odniesienia.

3.2. Inspiracja biologiczna

Mrówki komunikują się pośrednio, przez zmiany w otoczeniu (tzw. stygmergia). Idąca mrówka zostawia na podłożu feromon, a inne mrówki chętniej wybierają ścieżki, na których feromonu jest więcej. Na krótszej ścieżce mrówki szybciej wracają do mrowiska, więc w tym samym czasie przechodzi nią więcej mrówek i gromadzi się na niej więcej feromonu. Feromon z czasem paruje, przez co rzadziej używane, dłuższe ścieżki są stopniowo porzucane. W doświadczeniu z podwójnym mostkiem (Goss i in., 1989) kolonia mrówek argentyńskich w ten sposób wybierała krótszą z dwóch dróg do pożywienia.

Algorytm mrówkowy zaproponowany przez M. Dorigo przenosi ten mechanizm do optymalizacji: sztuczne mrówki budują rozwiązania krok po kroku, korzystając z feromonu odłożonego przez poprzednie pokolenia, a po każdej iteracji feromon jest aktualizowany zależnie od jakości znalezionych tras.

3.3. Budowa trasy przez mrówkę

Każda mrówka zaczyna w mrowisku. Będąc w punkcie i, wybiera kolejny punkt j spośród jeszcze nieodwiedzonych (zbiór N_i^k) z prawdopodobieństwem:

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in N_i^k} [\tau_{il}]^\alpha [\eta_{il}]^\beta}, \qquad \eta_{ij} = \frac{1}{d_{ij}} \tag{2}$$

gdzie τ_{ij} to ilość feromonu na krawędzi (i, j), η_{ij} to „widoczność", czyli odwrotność odległości, a α i β określają, jak mocno mrówka kieruje się feromonem, a jak mocno odległością. Wybór realizowany jest metodą ruletki. Po odwiedzeniu wszystkich punktów mrówka wraca do mrowiska.

3.4. Aktualizacja feromonu

Po zbudowaniu tras przez wszystkie m mrówek feromon najpierw paruje, a następnie każda mrówka odkłada feromon na krawędziach swojej trasy. Dodatkowo wzmacniana jest najlepsza dotąd znaleziona trasa (strategia elitarna):

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^k + e \cdot \Delta\tau_{ij}^{best} \tag{3}$$

$$\Delta\tau_{ij}^k = \frac{Q}{L_k} \tag{4}$$

$\Delta\tau_{ij}^k$ jest niezerowe tylko dla krawędzi, którymi przeszła mrówka k; L_k to długość jej trasy, a Q to stała. Im krótsza trasa, tym więcej feromonu dostają jej krawędzie. $\Delta\tau_{ij}^{best}$ liczone jest w ten sam sposób dla najlepszej dotąd trasy, a e jest wagą wzmocnienia elitarnego.

Przykład liczbowy. Mrówka stoi w mrowisku, a feromon jest jeszcze wszędzie równy ($\tau = \tau_0$). Najbliższe punkty leżą w odległości 73 i 79. Przy $\beta = 3$ ich wagi mają się do siebie jak $(79/73)^3 \approx 1{,}27$,

więc bliższy punkt dostaje około 51% szans, a dalszy około 40% (pozostałe, odległe punkty dzielą resztę). Po 8 iteracjach na krawędzi do bliższego punktu jest już około 3 razy więcej feromonu niż na krawędzi do dalszego i proporcja zmienia się na 79% do 19%. Dokładnie te liczby pokazuje aplikacja w trybie „Wybór" (rysunki 5 i 6 w rozdziale 6.3).

3.5. Pseudokod

```
wejście: macierz odległości d, parametry m, α, β, ρ, Q, e, liczba iteracji T
τ ← τ0 dla wszystkich krawędzi,  τ0 = 1 / (n · L_NN)
najlepsza ← brak
powtarzaj T razy:
    dla każdej mrówki k = 1..m:
        trasa_k ← [0]                       # start w mrowisku
        dopóki są nieodwiedzone punkty:
            wybierz j z prawdopodobieństwem p_ij  (wzór 2)
            dodaj j do trasa_k
        L_k ← długość(trasa_k)              # z powrotem do mrowiska
        jeśli L_k < długość(najlepsza): najlepsza ← trasa_k
    τ ← (1 − ρ) · τ                         # parowanie
    dla każdej mrówki k: dodaj Q / L_k na krawędziach trasa_k
    dodaj e · Q / L_best na krawędziach najlepszej trasy
    τ ← max(τ, τ_min)                       # żadna ścieżka nie znika całkiem
zwróć najlepsza
```

3.6. Parametry algorytmu

Tabela 2. *Parametry algorytmu mrówkowego*

Symbol	Znaczenie	Domyślnie	Zakres w aplikacji
m	liczba mrówek w jednej iteracji	20	5–100
α	waga feromonu w regule wyboru	1,0	0–5
β	waga odległości (widoczności)	3,0	0–10
ρ	współczynnik parowania feromonu	0,3	0,01–0,9
Q	stała ilości odkładanego feromonu	1	stała
e	waga wzmocnienia najlepszej trasy	1	stała
T	liczba iteracji	100 (w eksperymentach)	bez limitu (Start/Pauza)

3.7. Szczegóły implementacji

- **Feromon początkowy** $\tau 0 = 1/(n \cdot L_{NN})$, gdzie L_{NN} to długość trasy najbliższego sąsiada; dzięki temu skala feromonu nie zależy od wielkości planszy.
- **Dolne ograniczenie feromonu** $\tau_{min} = \tau 0 \cdot 10^{-3}$ zapobiega całkowitemu zanikowi ścieżek. Jest to szczególnie ważne po zablokowaniu ścieżki, gdy kolonia musi znaleźć objazd.
- **Wagi wyboru** $[\tau]^{\alpha} \cdot [\eta]^{\beta}$ obliczane są raz na iterację dla całej macierzy (biblioteka NumPy), a mrówka losuje kolejny punkt przez sumę skumulowaną wag.
- **Praca krokowa**: klasa AntColony udostępnia metodę iterate(), która wykonuje jedną iterację. Iteracja jest podzielona na dwa etapy – construct_tours() (mrówki budują trasy, feromon się

nie zmienia) i update_pheromone(tours) (parowanie i odkładanie feromonu). Dzięki temu aplikacja może między tymi etapami pokazać animację mrówek idących po swoich trasach.

- **Podgląd reguły wyboru**: metoda probabilities(i, visited) zwraca wektor prawdopodobieństw p_{ij} liczony dokładnie tym samym wzorem, którego używają mrówki. Aplikacja rysuje go na planszy w trybie „Wybór".

3.8. Dotychczasowe wyniki w literaturze

- **Ant System** (Dorigo, Maniezzo, Colorni 1996) na małych instancjach, np. Oliver30 (30 miast), znajdował najlepsze znane trasy i dorównywał innym metodom ogólnego przeznaczenia. Na większych instancjach ustępował jednak wyspecjalizowanym algorytmom dla TSP. W tej samej pracy pokazano, że dodanie mrówek elitarnych (EAS) poprawia wyniki, o ile ich liczba nie jest zbyt duża.
- **Ant Colony System** (Dorigo, Gambardella 1997) na losowych instancjach z 50 miastami oraz instancjach testowych Oliver30, Eil50, Eil75 i KroA100 dawał wyniki lepsze lub równe algorytmowi genetycznemu, programowaniu ewolucyjnemu, wyżarzaniu symulowanemu i sieciom samoorganizującym się. W połączeniu z lokalnym przeszukiwaniem 3-opt był konkurencyjny z najlepszymi wówczas algorytmami genetycznymi na instancjach z biblioteki TSPLIB liczących nawet ponad tysiąc miast.
- **MAX–MIN Ant System** (Stützle, Hoos 2000) przez ograniczenie feromonu do przedziału $[\tau_{min}, \tau_{max}]$ ograniczył problem stagnacji. Na instancjach TSPLIB uzyskiwał lepsze wyniki niż Ant System i jego wcześniejsze rozszerzenia, a z lokalnym przeszukiwaniem należy do najskuteczniejszych wariantów ACO dla TSP.
- **Stan wiedzy** (Dorigo, Stützle 2004): algorytmy mrówkowe z lokalnym przeszukiwaniem dają trasy bardzo bliskie optymalnym, ale dla samego TSP najlepsze pozostają metody wyspecjalizowane – heurystyka Lina–Kernighana oraz metody dokładne. Program Concorde (Applegate i in.) rozwiązał dokładnie instancję z 85 900 miastami. Siłą ACO jest raczej uniwersalność: ten sam schemat stosuje się do problemów trasowania pojazdów, przydziału czy harmonogramowania, a także do problemów zmieniających się w czasie, jak zablokowanie ścieżki w naszym rozszerzeniu.

Wyniki z literatury wskazują dwa kierunki, które sprawdzamy w projekcie: połączenie ACO z lokalnym przeszukiwaniem (2-opt) oraz ograniczanie feromonu w celu uniknięcia stagnacji. Nasze wstępne wyniki (rozdział 7) są z literaturą zgodne – podstawowy wariant bez lokalnego przeszukiwania jest porównywalny z 2-opt, ale wyraźnie od niego wolniejszy.

4. Algorytmy porównawcze

4.1. Przegląd zupełny (brute force)

Algorytm sprawdza wszystkie permutacje punktów pożywienia i wybiera najkrótszą trasę, więc zawsze daje wynik optymalny. Implementacja pomija trasy będące odwróceniem już sprawdzonych (pierwszy odwiedzony punkt musi mieć mniejszy numer niż ostatni) i przerywa liczenie trasy, gdy jej częściowa długość przekroczy najlepszy dotąd wynik. Mimo tych usprawnień złożoność pozostaje rzędu O(n!), dlatego w aplikacji algorytm jest dostępny tylko do 10 punktów pożywienia.

4.2. Algorytm najbliższego sąsiada

Prosta heurystyka zachłanna: startując z mrowiska, zawsze idziemy do najbliższego jeszcze nieodwiedzonego punktu, a na końcu wracamy do mrowiska. Złożoność $O(n^2)$. Algorytm jest bardzo szybki, ale nie „patrzy w przyszłość", więc pod koniec trasy często musi wykonać długie przejścia.

4.3. Algorytm 2-opt

Algorytm lokalnego ulepszania trasy (Croes, 1958). Startuje od trasy najbliższego sąsiada i sprawdza pary krawędzi (a, b) oraz (c, d). Jeśli zastąpienie ich krawędziami (a, c) i (b, d) skraca trasę, odwraca fragment trasy między b i c. Zamiany wykonuje się, dopóki żadna nie daje poprawy. Wynikowa trasa nie ma przecinających się odcinków. Jedno przejście kosztuje $O(n^2)$.

4.4. Zestawienie

Tabela 3. *Porównanie cech zastosowanych algorytmów*

Algorytm	Rodzaj	Złożoność	Optimum
Przegląd zupełny	dokładny	$O(n!)$	zawsze
Najbliższy sąsiad	heurystyka konstrukcyjna (zachłanna)	$O(n^2)$	nie
2-opt	przeszukiwanie lokalne	$O(n^2)$ na przejście	nie (optimum lokalne)
Algorytm mrówkowy	metaheurystyka populacyjna (inteligencja roju)	$O(T \cdot m \cdot n^2)$	nie (zwykle blisko)

5. Implementacja

5.1. Technologie

- **Python 3.13** – język programowania (projekt działa od wersji 3.10),
- **Pygame 2.6** – interfejs graficzny i animacja,
- **NumPy** – obliczenia na macierzach odległości i feromonu,
- **Matplotlib** – wykresy z wynikami eksperymentów.

5.2. Struktura projektu

Tabela 4. *Pliki projektu*

Plik	Zawartość
algorytmy.py	macierz odległości, długość trasy, przegląd zupełny, najbliższy sąsiad, 2-opt, klasa AntColony
aplikacja.py	aplikacja Pygame: plansza, panel sterowania, rysowanie feromonu i tras, panel wzorów, prawdopodobieństwa wyboru, animacja mrówek, porównanie algorytmów, blokowanie ścieżek
porownanie.py	eksperymenty; zapisuje tabele CSV i wykresy PNG do katalogu wyniki/
uruchom.bat	uruchomienie aplikacji dwuklikiem w systemie Windows; przy pierwszym uruchomieniu tworzy środowisko wirtualne .venv i instaluje biblioteki
README.md, requirements.txt	instrukcja uruchomienia i lista wymaganych bibliotek
dokumentacja/	niniejszy dokument, prezentacja, rysunki oraz skrypty generujące (narzedzia/)

5.3. Najważniejsze elementy kodu

Wszystkie algorytmy działają na tej samej macierzy odległości tworzonej funkcją distance_matrix(points, blocked), która od razu uwzględnia zablokowane ścieżki. Trasa jest listą numerów punktów zaczynającą się od 0 (mrowisko); powrót do mrowiska jest domyślny.

Tabela 5. *Interfejs klasy AntColony*

Element	Opis
AntColony(dist, n_ants, alpha, beta, rho, q, elite, seed)	tworzy kolonię i inicjalizuje feromon wartością $\tau 0$
iterate()	jedna iteracja: budowa tras, parowanie, odłożenie feromonu, aktualizacja najlepszej trasy
construct_tours()	etap 1 iteracji: każda z m mrówek buduje trasę; zwraca listę tras
update_pheromone(tours)	etap 2 iteracji: parowanie, odłożenie feromonu, wzmocnienie najlepszej trasy
probabilities(i, visited)	prawdopodobieństwa p_{ij} wyboru kolejnego punktu przez mrówkę stojącą w i (wzór 2)
run(T)	wykonuje T iteracji i zwraca najlepszą trasę

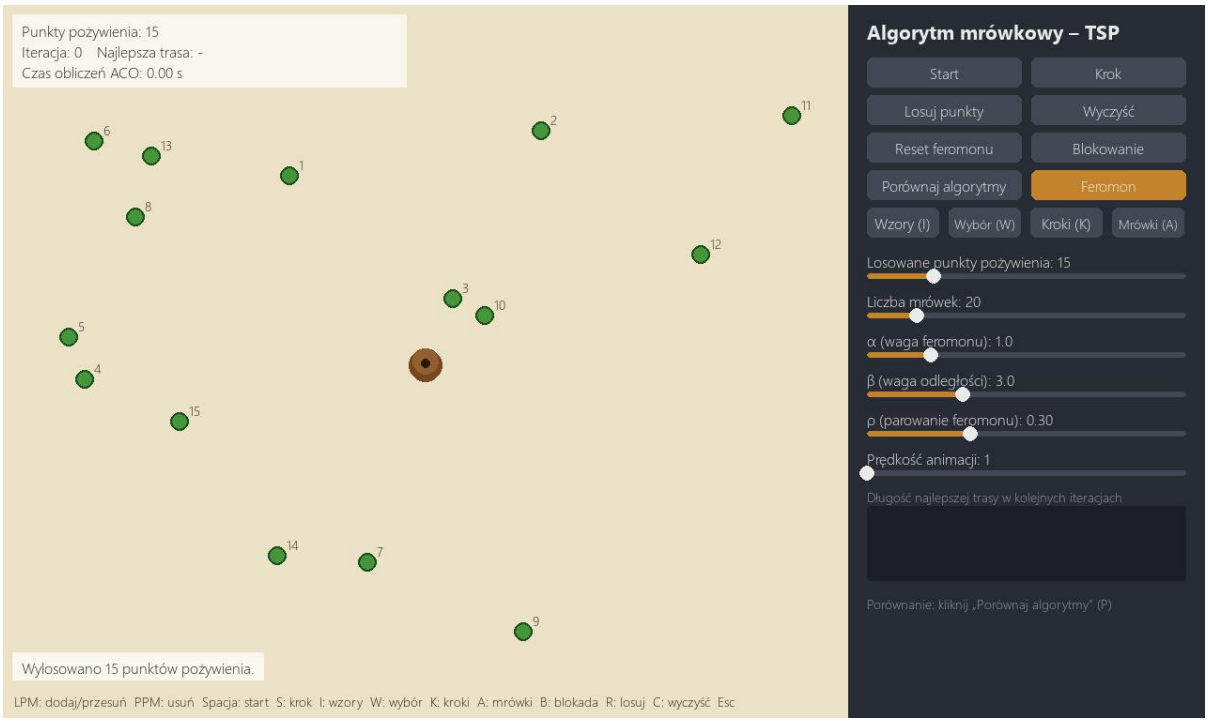
Element	Opis
set_distances(dist)	podmienia macierz odległości bez kasowania feromonu (używane po zablokowaniu ścieżki)
tau, best_tour, best_length, history	macierz feromonu, najlepsza trasa, jej długość i historia długości w kolejnych iteracjach

Aplikacja (klasa App) w każdej klatce obsługuje zdarzenia myszy i klawiatury, wykonuje zadaną liczbę iteracji ACO (gdy algorytm jest uruchomiony) i rysuje planszę. Grubość i przezroczystość linii między punktami są proporcjonalne do ilości feromonu na danej krawędzi, a najlepsza znaleziona trasa jest rysowana na czerwono. Parametry zmienione suwakami trafiają od razu do działającej kolonii, więc ich wpływ widać bez restartu.

6. Interfejs użytkownika – instrukcja obsługi

6.1. Okno aplikacji i sterowanie

Aplikację uruchamia się dwuklikiem na pliku **uruchom.bat** albo poleceniem **python aplikacja.py** (opcja **--okno** uruchamia ją w oknie zamiast na pełnym ekranie). Okno składa się z planszy (po lewej) i panelu sterowania (po prawej). Na starcie losowanych jest 15 punktów pożywienia, a mrowisko znajduje się na środku planszy.



Rysunek 1. *Okno aplikacji zaraz po uruchomieniu: mrowisko (brązowe), punkty z pożywieniem (zielone) i panel sterowania*

Tabela 6. *Sterowanie aplikacją*

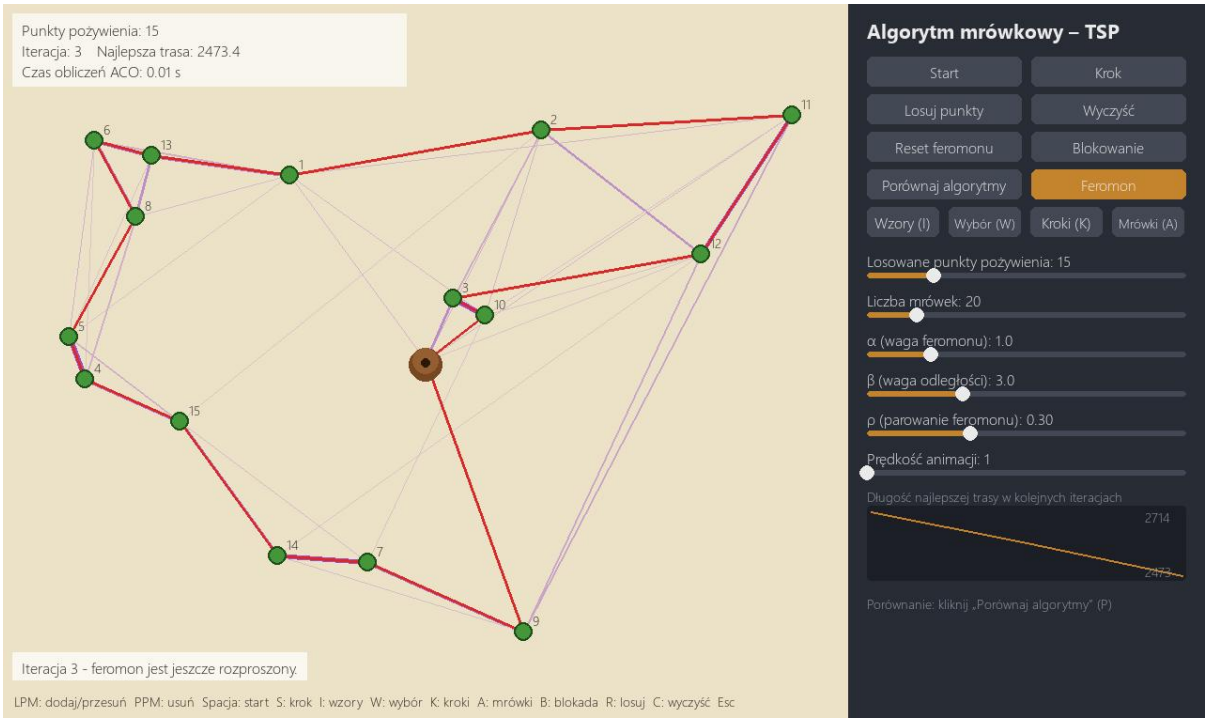
Akcja	Działanie
lewy przycisk myszy na pustym miejscu	dodaje punkt z pożywieniem
przeciągnięcie punktu	przesuwa punkt lub mrowisko
prawy przycisk myszy na punkcie	usuwa punkt (mrowiska nie można usunąć)
Spacja / „Start”	uruchamia lub zatrzymuje algorytm mrówkowy
S / „Krok”	wykonuje jedną iterację
R / „Losuj punkty”	losuje tyle punktów, ile ustawiono suwakiem
C / „Wyczyść”	usuwa wszystkie punkty pożywienia
„Reset feromonu”	zaczyna działanie kolonii od nowa
B / „Blokowanie”	w tym trybie kliknięcie odcinka blokuje lub odblokowuje ścieżkę
P / „Porównaj algorytmy”	wyznacza trasy wszystkimi algorytmami i pokazuje tabelę wyników; kliknięcie wiersza rysuje daną trasę
„Feromon”	włącza lub wyłącza rysowanie śladów feromonu

Akcja	Działanie
I / „Wzory (I)”	pokazuje lub ukrywa panel z grupą algorytmu, użytym wariantem, wzorami i bieżącymi parametrami
W / „Wybór (W)”	tryb podglądu reguły wyboru: kliknięcie punktu pokazuje prawdopodobieństwa przejścia do pozostałych punktów
K / „Kroki (K)”	w trybie „Wybór”: najbardziej prawdopodobna trasa mrówki krok po kroku z szansą każdego kroku
A / „Mrówki (A)”	włącza lub wyłącza animację mrówek przechodzących swoje trasy
suwaki	liczba losowanych punktów, liczba mrówek, α, β, ρ, prędkość animacji (iteracje na klatkę albo tempo ruchu mrówek)
F11 / Esc	przełącza pełny ekran i okno / zamyka aplikację

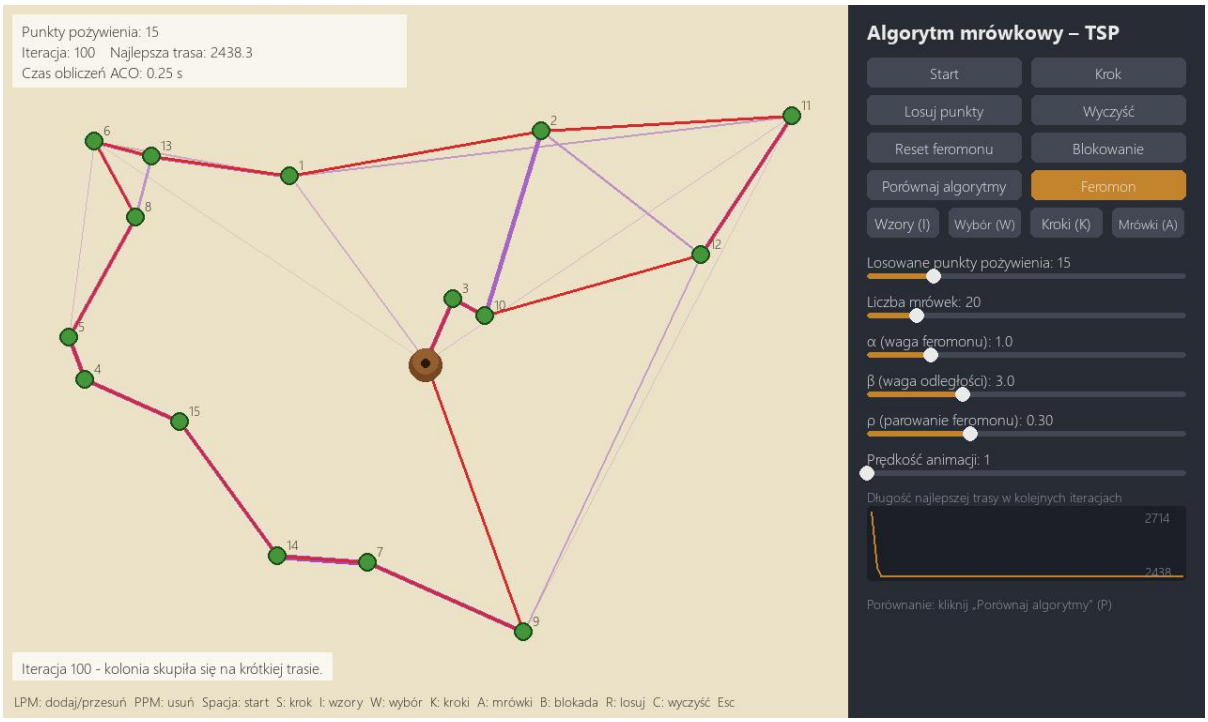
W lewym górnym rogu planszy wyświetlane są: liczba punktów, numer iteracji, długość najlepszej trasy i łączny czas obliczeń algorytmu mrówkowego. W panelu pod suwakami znajduje się wykres długości najlepszej trasy w kolejnych iteracjach.

Tabela 7. *Znaczenie elementów planszy*

Element	Znaczenie
brązowy kopczyk	mrowisko – początek i koniec każdej trasy
zielone kółka z numerami	punkty z pożywieniem
fioletowe linie	feromon; grubość i nieprzezroczystość linii rosną z ilością feromonu na krawędzi
czerwona linia	najlepsza trasa znaleziona do tej pory
czerwona przerywana linia ze znakiem X	zablokowana ścieżka
pomarańczowe linie z procentami	tryb „Wybór”: prawdopodobieństwo przejścia z zaznaczonego punktu
ciemne sylwetki na ścieżkach	tryb „Mrówki”: mrówki przechodzące swoje trasy w bieżącej iteracji



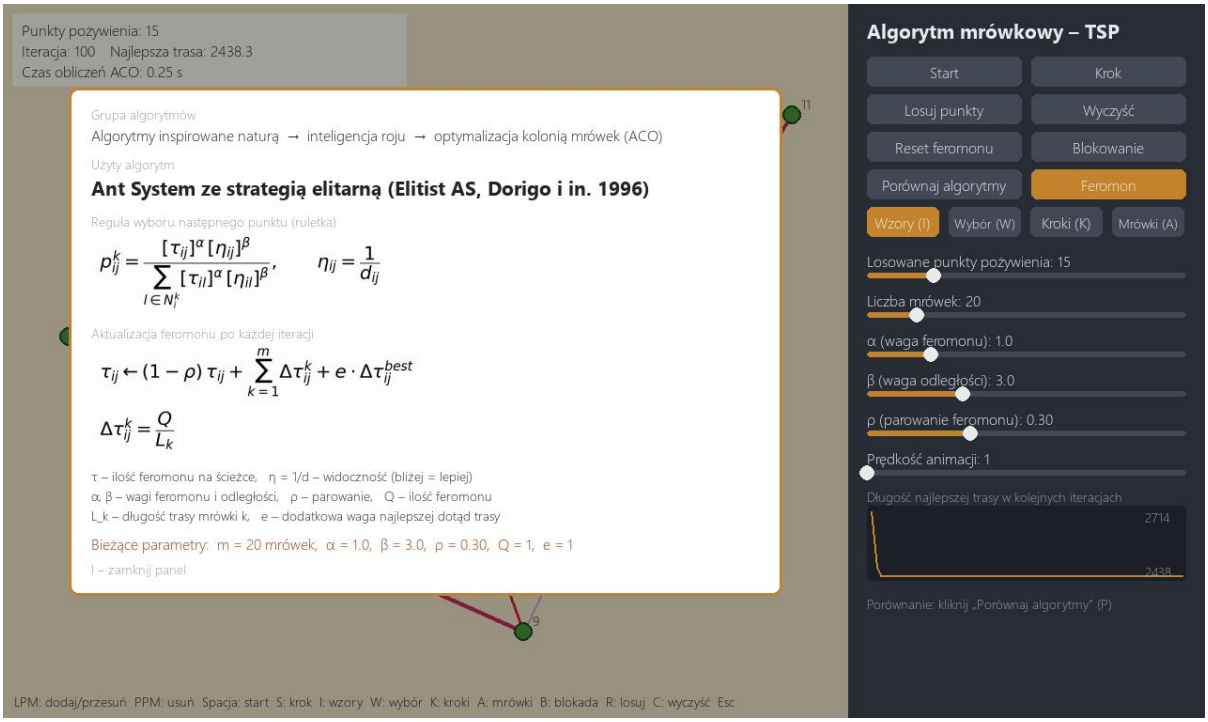
Rysunek 2. *Stan po 3 iteracjach: feromon (fioletowe linie) jest rozłożony na wielu ścieżkach*



Rysunek 3. *Stan po 100 iteracjach: feromon skupił się na krawędziach krótkiej trasy (czerwona), pozostałe ścieżki prawie zanikły*

6.2. Panel „Algorytm i wzory"

Klawisz **I** wyświetla na planszy panel, który zbiera w jednym miejscu najważniejsze informacje o algorytmie: grupę (algorytmy inspirowane naturą → inteligencja roju → ACO), użyty wariant (Ant System ze strategią elitarną), regułę wyboru kolejnego punktu, wzór aktualizacji feromonu, objaśnienie symboli oraz bieżące wartości parametrów. Wartości zmieniają się razem z suwakami, więc panel można zostawić otwarty podczas pokazu.



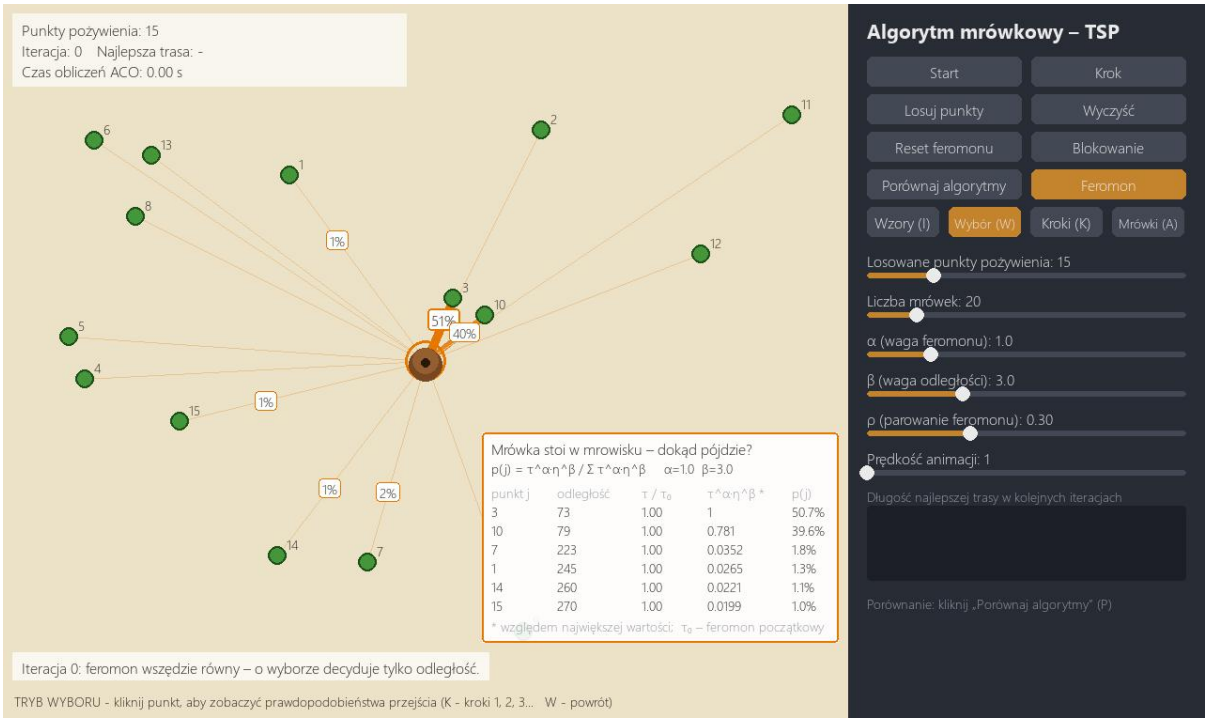
Rysunek 4. *Panel „Algorytm i wzory" z bieżącymi wartościami parametrów*

6.3. Podgląd reguły wyboru (tryb „Wybór")

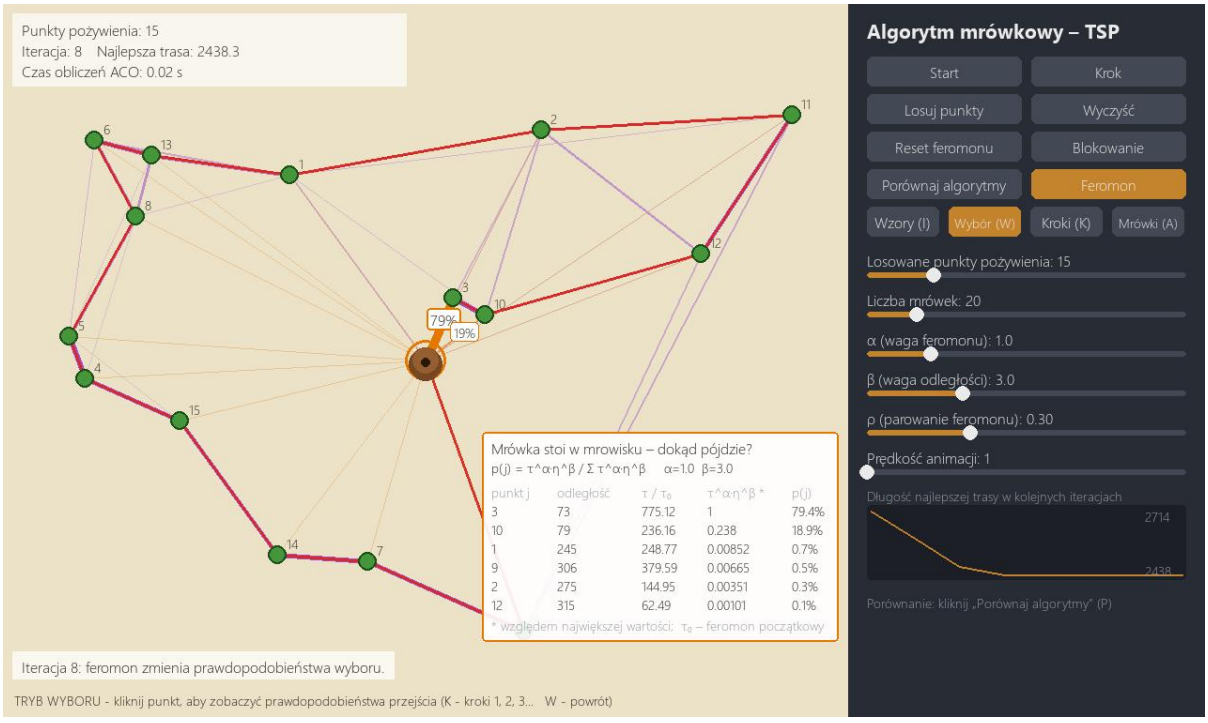
Tryb **W** pokazuje wzór (2) w działaniu. Po kliknięciu punktu aplikacja zakłada, że stoi w nim mrówka, i dla każdego nieodwiedzonego punktu liczy prawdopodobieństwo p_{ij}. Grubość pomarańczowej linii i etykieta pokazują to prawdopodobieństwo, a tabela obok rozpisuje wzór dla najbardziej prawdopodobnych punktów: odległość, ilość feromonu względem wartości początkowej τ/τ0, iloczyn $\tau^{\alpha} \cdot \eta^{\beta}$ i wynikowe p_{ij}.

Na początku (iteracja 0) feromon jest wszędzie równy, więc o wyborze decyduje tylko odległość – mrówka niemal na pewno pójdzie do jednego z dwóch najbliższych punktów. Po kilku iteracjach feromon zaczyna przeważać: jeden z dwóch podobnie odległych punktów dostaje prawie 80% szans. Przesuwając suwaki α i β, można na żywo zobaczyć, jak zmienia się decyzja mrówki – przy β = 0 wszystkie punkty są niemal równie prawdopodobne, a przy α = 0 feromon przestaje mieć znaczenie.

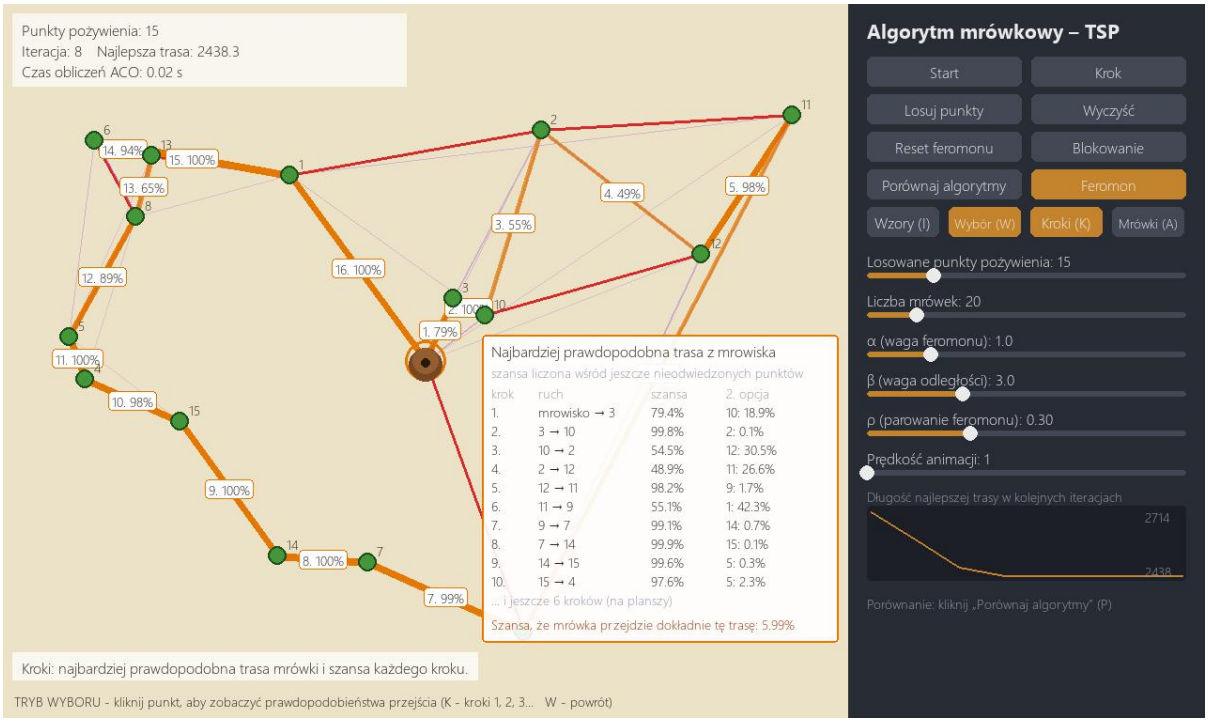
Przycisk **„Kroki (K)"** rozszerza ten podgląd na całą trasę. Mrówka na każdym kroku idzie do punktu o największym prawdopodobieństwie, a na planszy przy kolejnych odcinkach wyświetlane są etykiety z numerem kroku i szansą, np. „1. 79%", „2. 99%" (rysunek 7). Szansa każdego kroku liczona jest wzorem (2) tylko wśród punktów jeszcze nieodwiedzonych, więc pod koniec trasy rośnie aż do 100% (zostaje jeden punkt, a powrót do mrowiska jest wymuszony). Tabela podaje dla każdego kroku także drugą najlepszą opcję oraz szansę, że mrówka przejdzie dokładnie całą tę trasę (iloczyn szans wszystkich kroków).



Rysunek 5. *Tryb „Wybór" przed pierwszą iteracją: o prawdopodobieństwie decyduje tylko odległość*



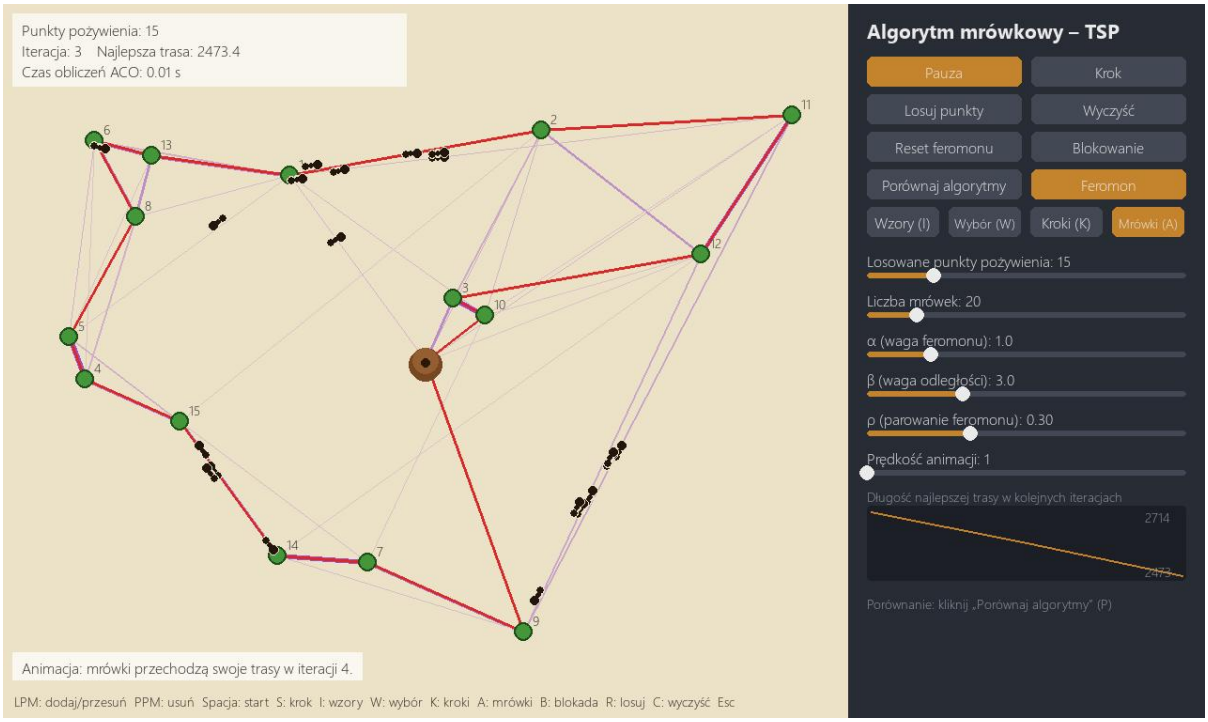
Rysunek 6. *Tryb „Wybór" po 8 iteracjach: feromon zwiększył szanse punktu 3 z 51% do 79%*



Rysunek 7. *Przycisk „Kroki": najbardziej prawdopodobna trasa mrówki z szansą każdego kroku*

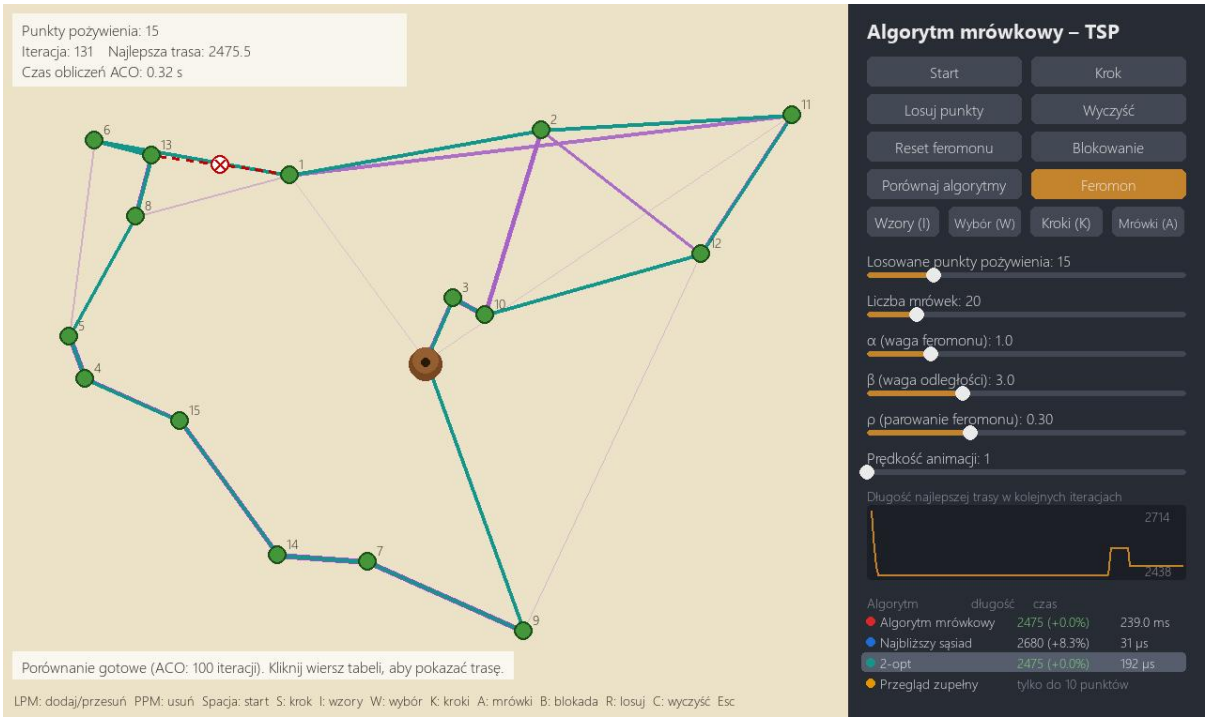
6.4. Animacja mrówek

W trybie **A** każda iteracja jest rozłożona na wiele klatek animacji. Najpierw mrówki budują trasy (metoda construct_tours()), potem przez kilka sekund przechodzą po nich na planszy (rysowanych jest najwyżej 30 mrówek), a dopiero gdy wszystkie wrócą do mrowiska, feromon paruje i jest odkładany (update_pheromone()). Odpowiada to kolejności z pseudokodu i pokazuje, że feromon zmienia się raz na iterację. Suwak prędkości zmienia tempo ruchu mrówek, a przycisk „Krok" odtwarza jedną iterację i zatrzymuje animację.



Rysunek 8. *Animacja mrówek: mrówki przechodzą swoje trasy w trakcie iteracji*

6.5. Porównanie algorytmów



Rysunek 9. *Tabela porównania algorytmów w panelu; na planszy pokazano trasę wyznaczoną algorytmem 2-opt. Czerwony znak X oznacza zablokowaną ścieżkę*

7. Badania i wyniki wstępne

7.1. Metodyka

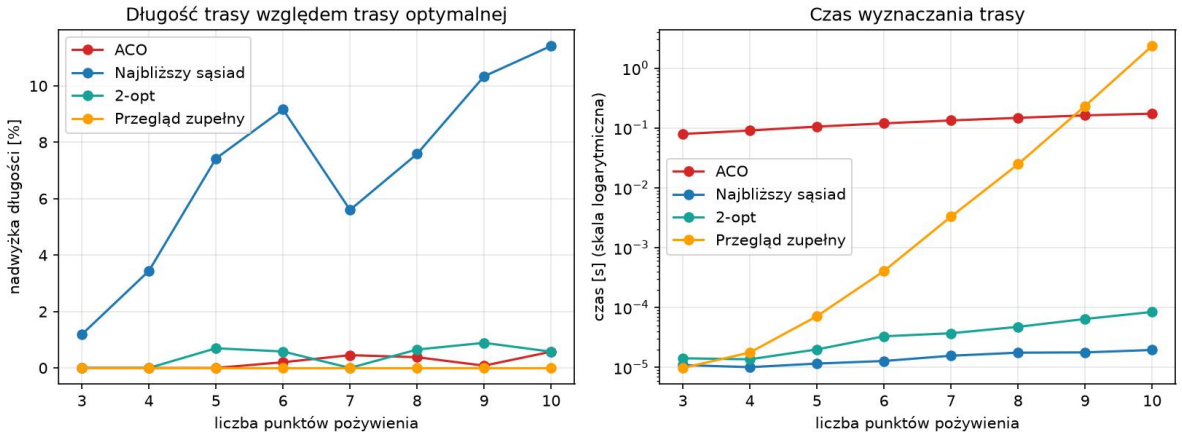
- Punkty (mrowisko i n punktów pożywienia) losowane są równomiernie w kwadracie 100 × 100; generator liczb losowych ma ustalone ziarno, dzięki czemu wyniki są powtarzalne.
- Algorytm mrówkowy: 20 mrówek, α = 1, β = 3, ρ = 0,3, 100 iteracji.
- Jakość trasy mierzona jest jako procentowa nadwyżka długości względem trasy optymalnej (małe instancje) lub najlepszej trasy znalezionej przez którąkolwiek z metod (duże instancje).
- Czas mierzony jest funkcją time.perf_counter() i obejmuje tylko obliczenia, bez rysowania.
- Wszystkie eksperymenty uruchamia skrypt **porownanie.py**; wyniki zapisuje do plików CSV i wykresów w katalogu wyniki/.

Testy przeprowadzono na komputerze z systemem Windows 11 i Pythonem 3.13. Czasy bezwzględne zależą od sprzętu, ale proporcje między algorytmami powinny być podobne na innych komputerach.

7.2. Małe instancje – porównanie z optimum

Tabela 8. *Małe instancje: nadwyżka długości trasy względem optimum [%] i czas [ms] (średnia z 5 instancji)*

n	ACO %	ACO ms	NN %	NN ms	2-opt %	2-opt ms	BF ms
3	0,00	79,4	1,19	0,011	0,00	0,014	0,010
4	0,00	90,8	3,44	0,010	0,00	0,014	0,018
5	0,00	105	7,43	0,012	0,70	0,020	0,072
6	0,20	119	9,17	0,013	0,58	0,033	0,409
7	0,45	133	5,60	0,016	0,00	0,037	3,3
8	0,38	148	7,59	0,018	0,65	0,048	24,8
9	0,08	162	10,34	0,018	0,89	0,065	233
10	0,58	174	11,42	0,020	0,58	0,085	2348



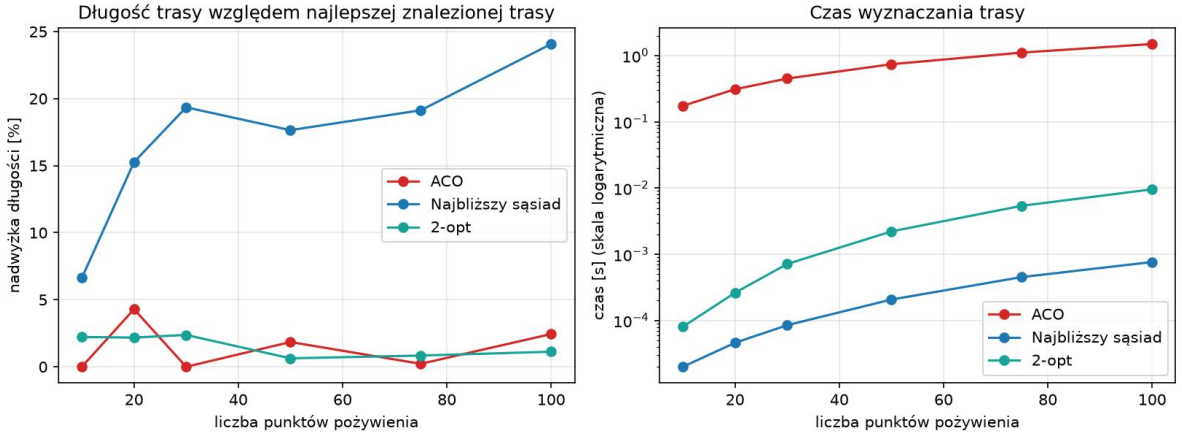
Rysunek 10. *Małe instancje: nadwyżka długości względem optimum (po lewej) i czas obliczeń w skali logarytmicznej (po prawej)*

Czas przeglądu zupełnego rośnie mniej więcej dziesięciokrotnie z każdym dodatkowym punktem: 233 ms dla 9 punktów i 2348 ms dla 10 punktów. Przy tym tempie dla 15 punktów obliczenia trwałyby kilka dni. Algorytm mrówkowy potrzebuje od 0,08 do 0,17 s, a jego czas rośnie z liczbą punktów bardzo powoli, i znajduje trasy dłuższe od optymalnej średnio o mniej niż 1%. Algorytm 2-opt daje podobną jakość w ułamku milisekundy, a najbliższy sąsiad jest najszybszy, ale jego trasy są dłuższe od optymalnych nawet o kilkanaście procent.

7.3. Duże instancje

Tabela 9. *Duże instancje: nadwyżka długości względem najlepszej znalezionej trasy [%] i czas [ms] (średnia z 3 instancji)*

n	ACO %	ACO ms	NN %	NN ms	2-opt %	2-opt ms
10	0,00	174	6,62	0,020	2,22	0,081
20	4,29	310	15,26	0,046	2,18	0,262
30	0,00	449	19,35	0,085	2,38	0,713
50	1,85	739	17,65	0,207	0,63	2,2
75	0,22	1108	19,13	0,452	0,84	5,4
100	2,44	1491	24,07	0,763	1,12	9,5



Rysunek 11. *Duże instancje (10–100 punktów): jakość trasy i czas obliczeń*

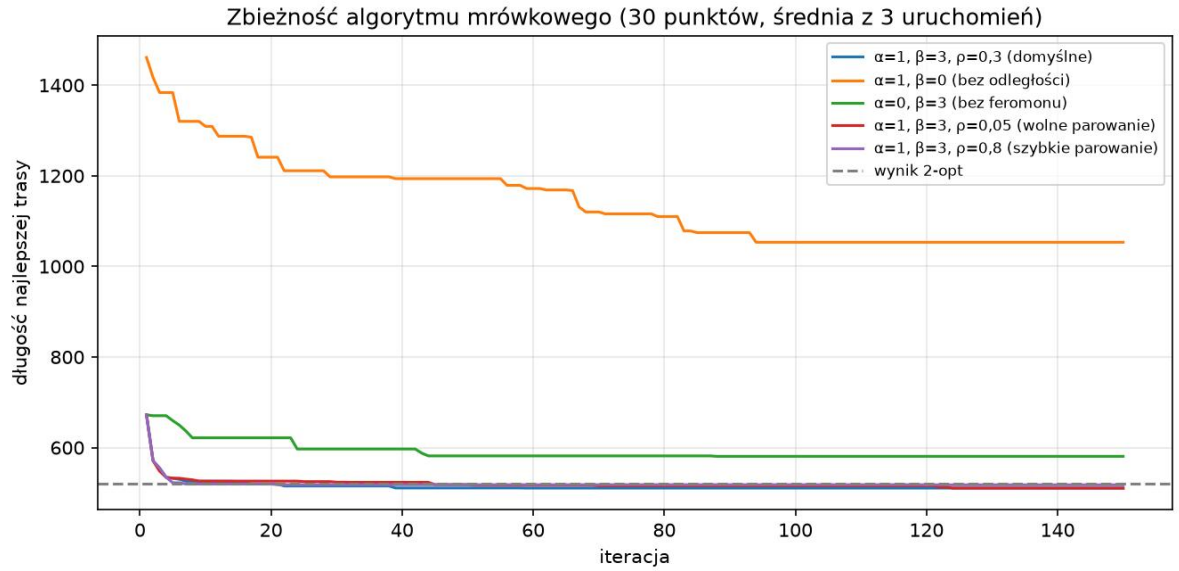
Dla większej liczby punktów różnica między najbliższym sąsiadem a pozostałymi metodami rośnie do około 15–24%. Algorytm mrówkowy i 2-opt dają trasy o podobnej jakości: w części instancji lepszy jest jeden, w części drugi, a różnice nie przekraczają kilku procent. Algorytm mrówkowy jest jednak znacznie wolniejszy (około 1,5 s dla 100 punktów przy 100 iteracjach), bo w każdej iteracji wiele mrówek buduje całe trasy.

7.4. Wpływ parametrów

Tabela 10. *Długość trasy po 150 iteracjach dla różnych parametrów (30 punktów, średnia z 3 uruchomień)*

Wariant parametrów	Długość	Względem 2-opt
α=1, β=3, ρ=0,3 (domyślne)	510,6	-1,9%
α=1, β=0 (bez odległości)	1053,3	+102,3%

Wariant parametrów	Długość	Względem 2-opt
α=0, β=3 (bez feromonu)	580,7	+11,5%
α=1, β=3, ρ=0,05 (wolne parowanie)	510,3	-2,0%
α=1, β=3, ρ=0,8 (szybkie parowanie)	517,8	-0,6%

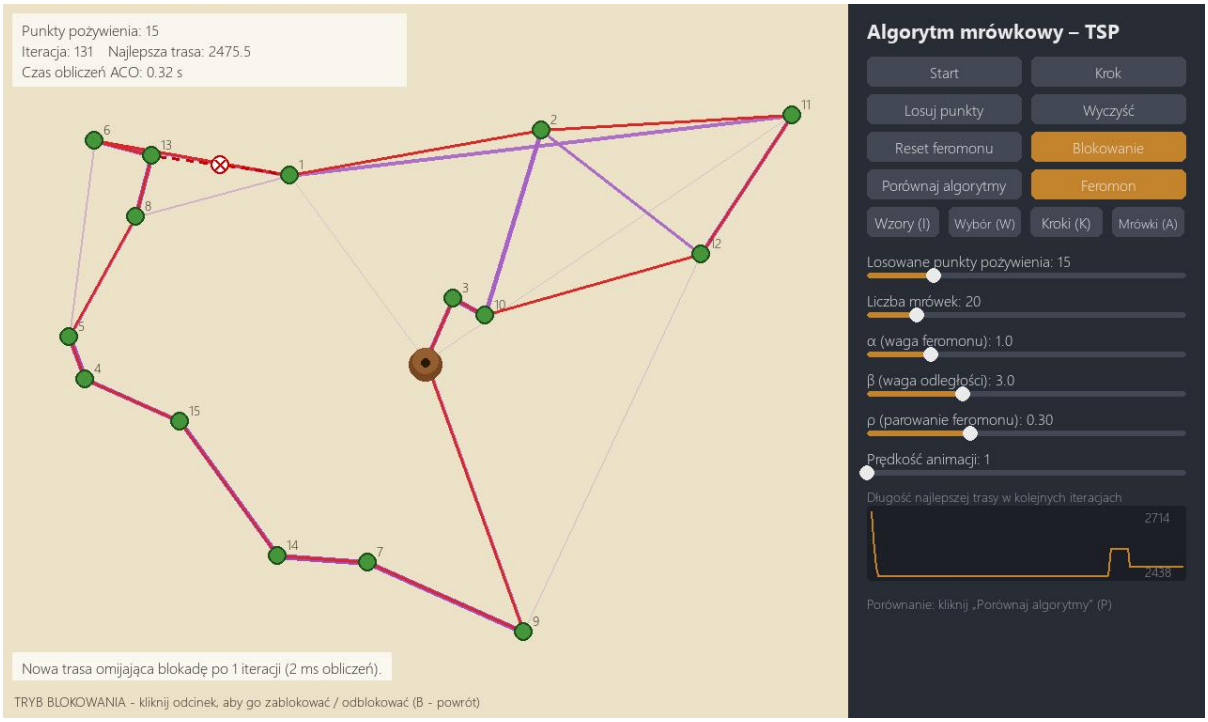


Rysunek 12. *Długość najlepszej trasy w kolejnych iteracjach dla różnych ustawień parametrów*

Najważniejsza jest informacja o odległości: przy β = 0 mrówki wybierają kolejne punkty niemal losowo i po 150 iteracjach trasa jest dwukrotnie dłuższa niż przy ustawieniach domyślnych. Przy α = 0 feromon nie ma znaczenia, więc algorytm sprowadza się do losowej heurystyki zachłannej, która nie uczy się na wcześniejszych trasach; wynik jest o około 11% gorszy niż 2-opt. Współczynnik parowania ρ w badanym zakresie wpływał na wynik w niewielkim stopniu; przy szybkim parowaniu (ρ = 0,8) kolonia szybciej się ustala i kończy na nieco dłuższej trasie.

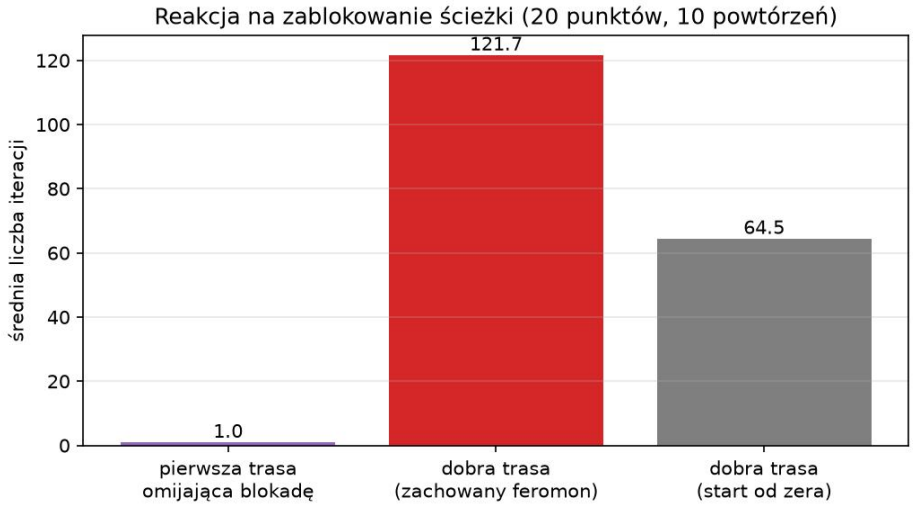
7.5. Rozszerzenie: zablokowanie ścieżki

W aplikacji można w trakcie działania algorytmu zablokować dowolną ścieżkę (tryb „Blokowanie”). Kolonia zachowuje wtedy dotychczasowy feromon, a aplikacja wyświetla, po ilu iteracjach znaleziono trasę omijającą blokadę.



Rysunek 13. *Kolonia po zablokowaniu jednego z odcinków najlepszej trasy (czerwony znak X) znalazła trasę zastępczą*

W eksperymencie (20 punktów, 10 powtórzeń) kolonia przez 100 iteracji szukała trasy, potem blokowano losową krawędź najlepszej trasy. Mierzono, po ilu iteracjach kolonia znajduje trasę omijającą blokadę, a po ilu trasę dobrej jakości, czyli najwyżej o 3% dłuższą niż wynik 2-opt dla grafu z blokadą. Dla porównania ten sam cel mierzono dla kolonii uruchomionej od zera.



Rysunek 14. *Średnia liczba iteracji potrzebnych po zablokowaniu ścieżki*

Trasa omijająca blokadę pojawia się praktycznie od razu (średnio po 1,0 iteracji), bo dotychczasowa najlepsza trasa staje się bardzo długa i każda trasa bez zablokowanej krawędzi ją zastępuje. Dojście do trasy dobrej jakości trwało jednak średnio 121,7 iteracji przy zachowanym feromonie i tylko 64,5 iteracji przy starcie od zera. Wynik ten pokazuje zjawisko stagnacji: po 100 iteracjach feromon jest silnie skupiony na starej trasie, więc mrówki z trudem odkrywają inne rozwiązania. W dalszej części

projektu planujemy sprawdzić częściowe wyrównanie feromonu po zablokowaniu ścieżki oraz wariant MAX-MIN Ant System z górnym i dolnym ograniczeniem feromonu.

7.6. Wnioski wstępne

- Przegląd zupełny jest praktyczny tylko do około 10 punktów; służy jako punkt odniesienia dla pozostałych metod.
- Algorytm mrówkowy znajduje trasy bliskie optymalnym (dla małych instancji średnio poniżej 1% od optimum), a jego czas rośnie wielomianowo.
- Na losowych instancjach do 100 punktów podstawowy ACO daje wyniki porównywalne z 2-opt, ale jest od niego znacznie wolniejszy. Dobrym kierunkiem jest połączenie obu metod (ulepszanie tras mrówek algorytmem 2-opt).
- Kluczowe dla działania ACO jest uwzględnienie odległości ($\beta > 0$); feromon ($\alpha > 0$) poprawia wynik względem samej heurystyki.
- Po zablokowaniu ścieżki kolonia od razu omija blokadę, ale skupiony feromon spowalnia dojście do nowej dobrej trasy.

8. Plan dalszych prac

Tabela 11. *Harmonogram prac*

Etap	Zakres	Stan
Zajęcia 1	wybór tematu, wstępny opis projektu	zrobione
Zajęcia 2	szczegółowy opis, klasyfikacja algorytmu i wyniki z literatury, dokumentacja, prezentacja; program z wizualizacją wzorów, podglądem prawdopodobieństw i animacją mrówek	zrobione (ten dokument)
Do zajęć 3	ACO z ulepszaniem tras algorytmem 2-opt; wariant MAX-MIN Ant System; badanie częściowego resetu feromonu po blokadzie	planowane
Do zajęć 3	więcej powtórzeń eksperymentów, odchylenie standardowe na wykresach, instancje z biblioteki TSPLIB (np. berlin52)	planowane
Do zajęć 3	tryb krok po kroku z podziałem iteracji na etapy; zapis i wczytywanie planszy	planowane
Zajęcia końcowe	ostateczna wersja programu, pełne wyniki, wnioski, prezentacja końcowa	planowane

Bibliografia

[1] M. Dorigo, T. Stützle, *Ant Colony Optimization*, MIT Press, Cambridge 2004.

[2] M. Dorigo, *Optimization, Learning and Natural Algorithms*, praca doktorska, Politecnico di Milano, 1992.

[3] B. Bullnheimer, R. F. Hartl, C. Strauss, *A new rank based version of the Ant System: a computational study*, Central European Journal for Operations Research and Economics, 7(1), 1999, s. 25–38.

[4] M. Dorigo, V. Maniezzo, A. Colorni, *Ant System: optimization by a colony of cooperating agents*, IEEE Transactions on Systems, Man, and Cybernetics – Part B, 26(1), 1996, s. 29–41.

[5] M. Dorigo, L. M. Gambardella, *Ant Colony System: a cooperative learning approach to the traveling salesman problem*, IEEE Transactions on Evolutionary Computation, 1(1), 1997, s. 53–66.

[6] T. Stützle, H. H. Hoos, *MAX–MIN Ant System*, Future Generation Computer Systems, 16(8), 2000, s. 889–914.

[7] S. Goss, S. Aron, J.-L. Deneubourg, J. M. Pasteels, *Self-organized shortcuts in the Argentine ant*, Naturwissenschaften, 76, 1989, s. 579–581.

[8] G. A. Croes, *A method for solving traveling-salesman problems*, Operations Research, 6(6), 1958, s. 791–812.

[9] D. L. Applegate, R. E. Bixby, V. Chvátal, W. J. Cook, *The Traveling Salesman Problem: A Computational Study*, Princeton University Press, 2006.

[10] Dokumentacja biblioteki Pygame, https://www.pygame.org/docs/